

# CS221 Spring 2024: Artificial Intelligence:

## Principles and Techniques

### Homework 3: Route Planning

SUNet ID: [your SUNet ID]  
Name: [your first and last name]  
Collaborators: [list all the people you worked with]

*By turning in this assignment, I agree by the Stanford honor code and declare that all of this is my own work.*

In route planning, the objective is to find the best way to get from point A to point B (think Google Maps). In this homework, we will build on top of the classic shortest path problem to allow for more powerful queries. For example, not only will you be able to explicitly ask for the shortest path from the Gates Building to the Green Library Coupa Cafe, but you can ask for the shortest path from Gates back to your dorm, stopping by the package center, gym, and the dining hall (in any order!) along the way.

**Before you get started, please read the Assignments section on the course website thoroughly.**

#### Problem 0: Grid City

Consider an infinite city consisting of locations  $(x, y)$  where  $x, y$  are integers. From each location  $(x, y)$ , one can go east, west, north, or south. These movements are what we refer to as ‘actions’. Specifically, an action is a change in the  $(x, y)$  coordinates:  $(+1, 0)$  represents moving east,  $(-1, 0)$  represents moving west,  $(0, +1)$  represents moving north, and  $(0, -1)$  represents moving south.

You start at  $(0, 0)$  and want to go to  $(m, n)$ , where  $m, n \geq 0$ . We can define the following search problem to capture this:

1.  $s_{\text{start}} = (0, 0)$
2.  $\text{Actions}(s) = \{(+1, 0), (-1, 0), (0, +1), (0, -1)\}$
3.  $\text{Succ}(s, a) = s + a$
4.  $\text{Cost}((x, y), a) = 1 + \max(x, 0)$  (i.e., it is more expensive as  $x$  increases)

5.  $\text{IsEnd}(s) = \mathbf{1}[s = (m, n)]$

1. What is the minimum cost of reaching location  $(m, n)$  (note that  $m, n \geq 0$ ) starting from location  $(0, 0)$  in the above city? Describe one possible path achieving the minimum cost. Is it unique (i.e., are there multiple paths that achieve the minimum cost)?

**What we expect:** Provide an expression for the minimum cost. In addition, please provide 1 - 2 sentences describing one possible minimum cost path and state whether it is unique.

**Your Solution:**

$$\frac{m(m+1)}{2} + n$$

The path is not unique, but the minimum cost is. Multiple paths can achieve the minimum cost, you can interleave movements (e.g., move east, then north, then east, then north), but the total number of east and north moves will be the same.

2. How will Uniform Cost Search (UCS) behave on this problem? Mark the following as true or false:
  - (a) UCS will never terminate because the number of states is infinite.
  - (b) UCS will return the minimum cost path and explore only locations between  $(0, 0)$  and  $(m, n)$ ; that is,  $(x, y)$  such that  $0 \leq x \leq m$  and  $0 \leq y \leq n$
  - (c) UCS will return the minimum cost path and explore only locations whose past costs are less than or equal to the minimum cost from  $(0, 0)$  to  $(m, n)$ .

**What we expect:** T/F for each subpart.

**Your Solution:** F; F; T

3. Now consider running UCS on an arbitrary graph. Mark the following as true or false:
  - (a) If you add a connection between two locations, the minimum cost cannot go up.
  - (b) If you make the cost of an action from some state small enough (possibly negative), that action will show up in the minimum cost path.
  - (c) If you increase the cost of each action by 1, the minimum cost path does not change (even though its cost does).

**What we expect:** T/F for each subpart.

**Your Solution:** T; F; T

## Problem 1: Finding Shortest Paths

We first start out with the problem of finding the shortest path from a start location (e.g., the Gates Building) to some end location. In Google Maps, you can only specify a specific end location (e.g., Coupa Cafe at Green Library).

In this problem, we want to give the user the flexibility of specifying multiple possible end locations by specifying a set of "tags" (e.g., so you can say that you want to go to any place with food versus a specific location like Tressider).

1. Implement `ShortestPathProblem` so that given a `startLocation` and `endTag`, the minimum cost path corresponds to the shortest path from `startLocation` to any location that has the `endTag`.

In particular, you need to implement `startState()`, `isEnd(state)`, `actionSuccessorsAndCosts(state)`. Note: please read the methods in 'util.py' (aside from 'PriorityQueue') for details.

Recall the separation between search problem (modeling) and search algorithm (inference). You should focus on modeling (defining the `ShortestPathProblem`); the default search algorithm, `UniformCostSearch` (UCS), is implemented for you in `util.py`.

**What we expect:** An implementation of the `ShortestPathProblem` class.

2. Run `python mapUtil.py > readableStanfordMap.txt` to write a (long-ish) file of the possible locations on the Stanford map along with their tags. Each tag is a `[key]=[value]`. Here are some examples of keys:

- `landmark`: Hand-defined landmarks (from `data/stanford-landmarks.json`)
- `amenity`: Various amenity types (e.g., `"parking_entrance"`, `"food"`)
- `parking`: Assorted parking options (e.g., `"underground"`)

Choose a starting location and end tag (perhaps that's relevant to your daily life) and implement `getStanfordShortestPathProblem()` to create a search problem. Then, run `python grader.py lb-custom` to generate `path.json`. Once generated, run `python visualization.py` to visualize it (opens in browser). Try different start locations and end tags. Pick two settings corresponding to the following:

- A start location and end tag that produced new insight into traveling around campus. Describe whether the system was useful.

A start location and end tag where the minimum cost path found isn't desirable. Is this due to incorrect modeling assumptions? Explain.

You should feel free to add new landmarks. If you are not familiar with the Stanford campus, please feel free to use the Campus Map website to learn more about buildings, amenities, and paths around campus.

### Your Solution:



- In particular, what *negative externalities* might result from this system being widely deployed? Please refer to these brief articles for inspiration: "Why Traffic Apps Make Congestion Worse," "The Perfect Selfishness of Mapping Apps", as well as the module on Externalities (video and pdf).

Discuss the impact of these externalities on users of your system and other people. Remember that these sorts of problems arise from the mismatch between the real world and one's model of it.

What are potential ways you could reduce this mismatch?

**What we expect:** Provide 3-5 sentences describing (a) two externalities (one for users of the route-planning system and one for other people/non-users of the system), as well as (b) at least one potential solution to reduce the impact of these problems.

### **Your Solution:**

#### **Externalities**

**For Users of the Route-Planning System:** Users may experience increased travel time and frustration due to congestion caused by others using the same system. As more individuals rely on the same optimized routes, the model's recommendations could lead to bottlenecks, especially at popular intersections or pathways. This collective behavior could undermine the initial goal of optimizing travel efficiency, as the influx of users overwhelms the designed pathways.

**For Non-Users of the System:** Non-users might face disruptions in their daily activities due to the increased traffic flow in certain areas that are recommended by the route-planning system. For instance, residents or individuals who typically travel along less busy routes may encounter unexpected congestion, leading to a degradation in their quality of life. This could also cause safety concerns, as increased traffic in pedestrian areas may lead to accidents or conflicts between vehicles and pedestrians.

#### **Potential Solutions**

One potential solution to reduce these externalities is to implement a dynamic routing system that adjusts recommended routes based on real-time traffic conditions and usage patterns. By incorporating data analytics and machine learning, the system could redistribute traffic more evenly across campus, avoiding over-saturation of any single route. Additionally, providing alternative route options or encouraging off-peak travel could help mitigate congestion and distribute the travel load more effectively, minimizing the negative impacts on both users and non-users.

## Problem 2: Finding Shortest Paths with Unordered Waypoints

Let's introduce an even more powerful feature: unordered waypoints! In Google Maps, you can specify an ordered sequence of waypoints that a path must go through – for example, going from point A to point X to point Y to point B, where [X, Y] are "waypoints" (such as a gas station or a friend's house).

However, we want to consider the case where the waypoints are unordered: X, Y, so that both  $A \rightarrow X \rightarrow Y \rightarrow B$  and  $A \rightarrow Y \rightarrow X \rightarrow B$  are allowed. Moreover, X, Y, and B are each specified by a tag like in Problem 1 (e.g., amenity=food).

This is a neat feature if you think about your day-to-day life; you might be on your way home after a long day, but need to stop by the package center, Tressider to grab a bite of food, and the bookstore to buy some notebooks. Having the ability to get a short, quick path that hits all these stops might be really convenient (rather than searching over the various waypoint orderings yourself).

1. Implement `WaypointsShortestPathProblem` so that given a `startLocation`, set of `waypointTags`, and an `endTag`, the minimum cost path corresponds to the shortest path from `startLocation` to a location with the `endTag`, such that all of `waypointTags` are covered by some location in the path.

Note that a single location can be used to satisfy multiple tags.

Like in Problem 1, you need to implement `startState()`, `isEnd(state)`, and `actionSuccessorsAndCosts(state)`.

There are many ways to implement this search problem, so you should think carefully about how to design your `State`. We want to optimize for a compact state space so that search is as efficient as possible.

**What we expect:** An implementation of the `WaypointsShortestPathProblem` class. To get full credit, your implementation must have the right asymptotic dependence on the number of locations and waypoints. Note that your code will timeout if you do this incorrectly.

2. If there are  $n$  locations and  $k$  waypoint tags, what is the maximum possible number of states given a suitable state definition for part a that UCS could visit?

**What we expect:** A mathematical expression that depends on  $n$  and  $k$ , with a brief explanation justifying it.

**Your Solution:**

$$n \times 2^k$$

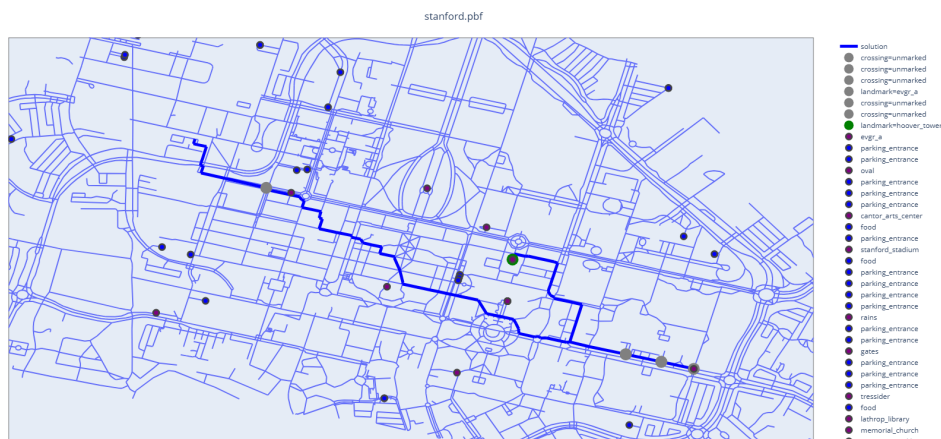
$n$  is the number of locations: Each unique location on the map represents a distinct state of the search.  $k$  is the number of waypoint tags: Each waypoint tag can either be

marked as visited (1) or not visited (0). Since there are  $k$  tags, the memory state can represent  $2^k$  different combinations of visited and unvisited tags. By combining every possible location with each combination of waypoint tags, we get a total of  $n \times 2^k$  unique states.

3. Choose a starting location, set of waypoint tags, and an end tag (perhaps that captures an interesting route planning problem relevant to you), and implement `getStanfordWaypointsShortestPathProblem()` to create a search problem. Then, similar to Problem 1b, run `python grader.py 2c-custom` to generate `path.json`. Once generated, run `python visualization.py` to visualize it (opens in browser).

**What we expect:** A screenshot of the visualized path with a list of the waypoint tags you selected. In addition, provide one or more sentences describing unexpected or interesting features of the selected path. Does it match your expectations? What does this path fail to capture that might be important?

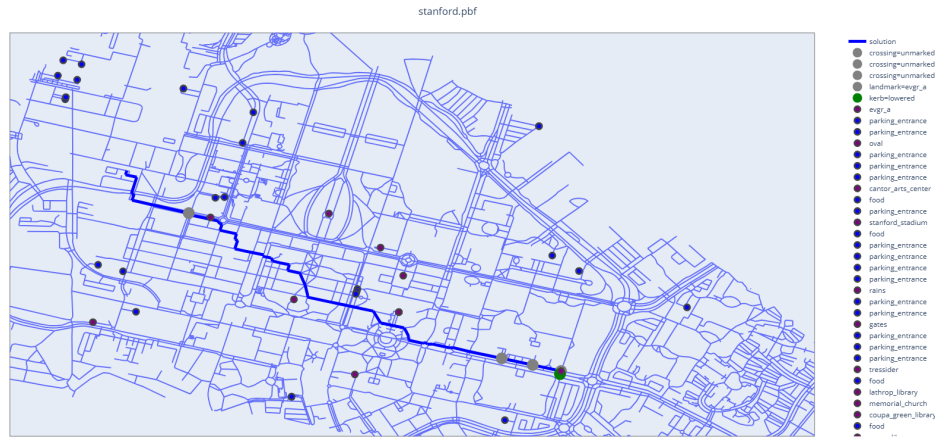
### Your Solution:



4. Ride sharing companies use route-finding systems similar to the one you built in this problem to route their drivers. However, these systems have been criticized for using exploitative behavioral nudges. For example, a New York Times article reported how one app uses “forward dispatch,” a feature that dispatches a new ride to a driver before the current one ends, to constantly keep drivers busy. This makes it more difficult for drivers to take breaks.

We want you to discuss how these companies could implement better labor practices, finding a way to use the unordering waypoints feature to balance company goals with





the protection of driver interests. Specifically, we want you to think about ways your new unordered waypoints feature could help drivers in their working experiencing.

What waypoints could you include from one drop-off location to the next pick-up location to improve the driver experience? What information about drivers would you need to determine appropriate waypoints?

Note, however, that the unordered waypoints feature is an example of a dual use technology. For more, refer to the module on dual use technologies to help answer this question video and pdf).

Thus, we also want you to answer: What are some ways that a company could use this waypoints feature to increase profits at the expense of driver health? What are any downsides in collecting the appropriate information you need to recommend the waypoints?

**What we expect:** Provide 3-5 sentences describing (a) the waypoints you would include to improve the driver experience, (b) at least one dimension of drivers' identity that could inform selection of waypoints, and (c) a potential negative use of the waypoints feature that a company could use to increase profits while harming the driver experience or driver health.

### Your Solution:

**(a) Waypoints to Improve Driver Experience:** To enhance driver experience, I would include waypoints that lead to rest areas, gas stations, or food outlets along the route from one drop-off location to the next pick-up location. These waypoints would provide drivers with the opportunity to take breaks, refuel, and eat, thereby reducing fatigue and improving their overall well-being. Including rest stop waypoints at appropriate intervals based on driving time and distance would encourage healthier driving habits.

**(b) Dimension of Drivers' Identity:** One crucial dimension of drivers' identity that could inform waypoint selection is their driving patterns or preferences, such as whether they prefer to work longer shifts or shorter bursts. For instance, some drivers might favor longer driving periods followed by extended breaks, while others may prefer shorter shifts with more frequent breaks. Collecting data on individual driver preferences and patterns can help tailor the route to better suit their needs.

**(c) Potential Negative Use of Waypoints Feature:** However, the same unordered waypoints feature could be exploited by the company to increase profits at the expense of driver health. For example, the company could prioritize waypoints that lead to areas with high demand but limited rest opportunities, thus pushing drivers to work longer without breaks. This might enhance ride completion rates and increase earnings for the company, but it could lead to driver burnout and health issues over time. Additionally, collecting and analyzing the necessary data to personalize routes may raise privacy concerns among drivers, as they might feel their work patterns and preferences are being monitored too closely.

### Problem 3: Speeding up Search with A\*

In this problem, we will explore how to speed up search by reducing the number of states that need to be expanded using various A\* heuristics.

1. In this problem, you will implement A\* to speed up the search. Recall that in class, A\* is just UCS with a different cost function (a new search problem), so we will implement A\* via a reduction to UCS.

In particular, you should implement `aStarReduction()` which takes a search problem and a heuristic as input and returns a new search problem.

**What we expect:** An implementation of the `NewSearchProblem` class in the `aStarReduction(problem, heuristic)` function. As in prior problems, you need to implement `startState()`, `isEnd(state)`, and `actionSuccessorsAndCosts(state)`.

2. We are now ready to speed up search by simply implementing various heuristics. First, implement `StraightLineHeuristic` for Problem 1, which returns the straight line distance from a state to any location with the end tag. Note: you might want to precompute some things so that evaluating the heuristic is fast.

**What we expect:** An implementation of the `StraightLineHeuristic` class.

3. Next, implement `NoWaypointsHeuristic` for Problem 2, which returns the minimum cost path from a state to any location with the end tag but ignoring all the waypoints (essentially the solution to Problem 1, so you can reuse that if you'd like). Note: you might want to precompute some things so that evaluating the heuristic is fast.

**What we expect:** An implementation of the `NoWaypointsHeuristic` class.

4. Recall that heuristics are most effective when they are equal to the future cost. Consider a  $n \times n$  grid map (`createGridMap`) where the start and end are at the opposite corners; for  $n = 10$ , we would have `startLocation = "0,0"`, `endTag = "label=9,9"`.

Provide a concrete example of  $n$  `waypointTags` so that running A\* with the `NoWaypointsHeuristic` results that does not require traversing the waypoints (i.e., a vanilla shortest paths problem).

**What we expect:** An example of  $n$  `waypointTags` that satisfies the constraints of the question.

**Your Solution:** `waypointTags = ["0,0", "1,1", ..., "n-1,n-1"]`

## Submission

Submission is done on Gradescope.

**Written:** When submitting the written parts, make sure to select **all** the pages that contain part of your answer for that problem, or else you will not get credit. To double check after submission, you can click on each problem link on the right side and it should show the pages that are selected for that problem.

**Programming:** After you submit, the autograder will take a few minutes to run. Check back after it runs to make sure that your submission succeeded. If your autograder crashes, you will receive a 0 on the programming part of the assignment. Note: the only file to be submitted to Gradescope is `submission.py`.

More details can be found in the Submission section on the course website.